

Design Patterns

Learn object oriented recipes to improve your code

```
interface Course {  
    Design evolve(Code code);  
    Tests protect(Behavior b);  
    Spec guide(AIAgent agent);  
}
```

Design Patterns

Reusable design experience for Java, modern frameworks and AI-assisted software engineering

From hard-coded code to flexible design

```
// Problem: creation is hard-coded
ReportExporter exporter = new PdfExporter();
exporter.export(report);

// Pattern-based design: depend on abstraction
ReportExporter exporter = factory.create(format);
exporter.export(report);
```

01 Creational

Who creates objects?

02 Structural

How are objects composed?

03 Behavioral

How do objects collaborate?

Learning goals

This block reviews classical GoF patterns and connects them with modern Java and AI workflows.

1 Recognize design forces

Identify repeated problems: variation, creation, coupling, notification, undo, state.

2 Use patterns critically

Apply patterns when they simplify evolution — not because they look elegant.

3 Read Java implementations

Understand interfaces, abstract classes, generics, lambdas and records inside pattern code.

4 Design with AI in mind

Write specs and tests so AI-generated changes respect architecture and intent.

Mental model

Pattern = Context + Problem + Forces + Solution + Consequences

Why patterns still matter

Patterns are not old recipes; they are a language for discussing trade-offs.

Without patterns

- “Just add an if” everywhere
- Concrete classes hard-coded
- Tests depend on real services
- AI assistant edits code blindly
- Architecture is implicit

With patterns

- Extension points are explicit
- Code depends on interfaces
- Tests use substitutes/mocks
- AI works against stable specs
- Architecture is reviewable

Design pressure

```
// Bad pressure:  
if (format.equals("pdf")) ...  
if (format.equals("json")) ...  
if (format.equals("html")) ...  
  
// Better pressure:  
interface Exporter {  
    void export(Report r);  
}
```

Patterns are especially useful when the code must be changed repeatedly by different humans, teams, frameworks — or AI agents.

What is a design pattern?

A named, reusable solution to a recurring design problem in a context.



Pattern thinking

Example: “We need several algorithms for pricing, selectable at runtime, without changing `OrderService`.”

→ Strategy pattern: put each algorithm behind the same interface.

Important: a pattern is not copy/paste code. It is a reusable micro-architecture with consequences.

How to document a pattern

Use a compact format that captures design intent, not only implementation.

Name

Shared vocabulary: Strategy, Observer, Factory...

Intent

What problem does it solve?

Motivation

A concrete example before the abstraction.

Structure

Participants and collaborations.

Consequences

Benefits, costs, complexity, limitations.

Known uses

Frameworks, libraries, existing codebases.

Course rule

When presenting a pattern in this course: show the problem first, then the pattern, then the trade-offs.

The GoF catalog: three families

The classical catalog remains a useful map, but we will teach the most relevant patterns deeply.

C Creational

- Factory Method
- Abstract Factory
- Builder
- Prototype
- Singleton

S Structural

- Adapter
- Bridge
- Composite
- Decorator
- Facade
- Flyweight
- Proxy

B Behavioral

- Observer
- Strategy
- Command
- Iterator
- State
- Template Method
- Visitor
- Mediator

Focus: patterns that appear in Java frameworks, testing, plugins, APIs, GUIs and AI-agent workflows.

Core design principle: composition over inheritance

Many patterns replace rigid inheritance with delegation through interfaces.

Inheritance

- Static relationship
- Good for true “is-a” hierarchy
- Can break encapsulation
- Base-class changes may affect subclasses
- Harder to change behavior at runtime

Composition + delegation

- Dynamic relationship
- Good for “has-a / uses-a” design
- Preserves encapsulation better
- Behavior can be swapped at runtime
- Requires more explicit interfaces

Delegation in one line

```
class Window { private Rectangle bounds; int area() { return bounds.area(); } }
```

Running example: a Maze game

The original TAP slides use a Maze example to motivate creational patterns.

Hard-coded creation

```
class MazeGame {  
    Maze createMaze() {  
        Maze maze = new Maze();  
        Room r1 = new Room(1);  
        Room r2 = new Room(2);  
        Door door = new Door(r1, r2);  
  
        maze.addRoom(r1);  
        maze.addRoom(r2);  
        r1.setSide(Direction.EAST, door);  
        r2.setSide(Direction.WEST, door);  
        return maze;  
    }  
}
```

Problem

Creation code knows all concrete classes.

Change request

Add BombedWall, EnchantedRoom, DoorNeedingSpell.

Pattern goal

Remove explicit references to concrete classes.

Factory Method

Let subclasses decide which concrete product to create.

Creator with factory methods

```
abstract class MazeGame {
    protected Room makeRoom(int n) { return new Room(n); }
    protected Wall makeWall() { return new Wall(); }
    protected Door makeDoor(Room a, Room b) {
        return new Door(a, b);
    }

    public Maze createMaze() {
        Room r1 = makeRoom(1);
        Room r2 = makeRoom(2);
        Door d = makeDoor(r1, r2);
        // common construction algorithm...
        return maze;
    }
}
```

Subclass selects product

```
class EnchantedMazeGame extends MazeGame {
    @Override protected Room makeRoom(int n) {
        return new EnchantedRoom(n, castSpell());
    }
}
```

Use when

The algorithm is stable but concrete products vary.
You want extension without modifying the base algorithm.

Modern Java angle: a factory method can return an interface, sealed hierarchy, record, or lambda-backed implementation.

Abstract Factory

Create families of related objects without specifying their concrete classes.

Family factory

```
interface MazeFactory {
    Maze makeMaze();
    Room makeRoom(int n);
    Wall makeWall();
    Door makeDoor(Room a, Room b);
}

class EnchantedMazeFactory implements MazeFactory {
    public Room makeRoom(int n) {
        return new EnchantedRoom(n, castSpell());
    }
    public Door makeDoor(Room a, Room b) {
        return new DoorNeedingSpell(a, b);
    }
}
```

Client depends only on factory

```
class MazeGame {
    Maze createMaze(MazeFactory f) {
        Maze maze = f.makeMaze();
        Room r1 = f.makeRoom(1);
        Room r2 = f.makeRoom(2);
        Door d = f.makeDoor(r1, r2);
        return maze;
    }
}
```

Good for

UI themes: Button + Window + Menu

Database families: Connection + Statement

Game variants: Room + Door + Wall

Creational

Factory in practice: JDBC connections and statements

JDBC hides vendor-specific implementation classes behind factory-style methods.

Factory roles

DriverManager is a static factory that selects a JDBC driver. A Connection then acts as a factory for Statements, PreparedStatements and CallableStatements.

Why this is useful

Client code depends on standard JDBC interfaces. The concrete classes come from PostgreSQL, MySQL, H2, Oracle, etc., but the application code does not instantiate them directly.

```
// DriverManager chooses the concrete driver
Connection con = DriverManager.getConnection(
    url, username, password);

// Connection creates JDBC objects
Statement st = con.createStatement();
PreparedStatement ps = con.prepareStatement(sql);
CallableStatement cs = con.prepareCall(procedure);

// Client sees interfaces, not vendor classes
System.out.println(con.getClass().getName());
```

Design idea: creation is centralized in a factory-style API, while the client only knows stable abstractions.

JDBC example: factory-created collaborators

A small complete example using try-with-resources and PreparedStatement.

```
String sql = ""
    SELECT id, name, grade
    FROM students
    WHERE grade >= ?
    ORDER BY grade DESC
    "";

try (Connection con = DriverManager.getConnection(url, user, pass);
    PreparedStatement ps = con.prepareStatement(sql)) {

    ps.setDouble(1, 5.0);

    try (ResultSet rs = ps.executeQuery()) {
        while (rs.next()) {
            long id = rs.getLong("id");
            String name = rs.getString("name");
            double grade = rs.getDouble("grade");
            System.out.printf("%d %s %.1f%n", id, name, grade);
        }
    }
}
```

Factory Method view

`con.prepareStatement(sql)` creates an object specialized for that SQL statement. The concrete implementation is provided by the driver.

Security note

Prefer PreparedStatement over string concatenation. It separates SQL code from parameter values and helps prevent SQL injection.

Pattern lesson

Factory APIs often appear in libraries: XML parsers, JDBC, HTTP clients, logging, JSON mappers and dependency-injection containers.

Builder

Separate the construction process from the representation produced.

Different representation

```
interface MazeBuilder {
    void buildMaze();
    void buildRoom(int n);
    void buildDoor(int from, int to);
    Maze result();
}

class CountingMazeBuilder implements MazeBuilder {
    private int rooms, doors;
    public void buildRoom(int n) { rooms++; }
    public void buildDoor(int a, int b) { doors++; }
    public Maze result() { return null; }
}
```

Same process

```
class MazeGame {
    Maze createMaze(MazeBuilder b) {
        b.buildMaze();
        b.buildRoom(1);
        b.buildRoom(2);
        b.buildDoor(1, 2);
        return b.result();
    }
}
```

Modern Java use

- Immutable objects with many optional fields
- Fluent APIs
- Test data builders
- Configuration objects

Builder in modern Java

Builders are common when constructors become unreadable or validation is needed.

Fluent builder

```
public final class ExportJob {
    private final String input;
    private final Format format;
    private final boolean compress;

    private ExportJob(Builder b) { this.input = b.input; this.format = b.format; this.compress = b.compress; }

    public static Builder builder(String input) { return new Builder(input); }

    public static final class Builder {
        private final String input;
        private Format format = Format.PDF;
        private boolean compress = false;
        private Builder(String input) { this.input = input; }
        public Builder format(Format f) { this.format = f; return this; }
        public Builder compress() { this.compress = true; return this; }
        public ExportJob build() { return new ExportJob(this); }
    }
}
```

But for small immutable data, prefer records: record ExportJob(String input, Format format, boolean compress) {}

Singleton: useful idea, dangerous default

Ensure one instance — but avoid turning it into hidden global state.

Safe simple singleton with enum

```
public final class Logger {
    private static final Logger INSTANCE = new Logger();

    private Logger() {
        // private constructor: no one can create another
        // Logger
    }

    public static Logger getInstance() { return INSTANCE; }

    public void info(String message) {
        System.out.println("[INFO] " + message); } }
```

Possible uses

Configuration, registries, shared immutable resources.

Risks

- Hidden dependencies
- Hard tests
- Global mutable state
- Concurrency bugs

Better testability

```
Logger log = Logger.getInstance();
log.info("Application started");
log.info("Loading configuration");
```

Prototype

Create new objects by copying an existing object.

Copy-based creation

```
interface MazePrototypeFactory {
    Room roomPrototype();
    Door doorPrototype();

    default Room makeRoom(int n) {
        Room r = roomPrototype().copy();
        r.initialize(n);
        return r;
    }
}

// Prototype is useful when object creation is expensive
// or objects are configured dynamically.
```

Modern Java caution

`clone()` is rarely recommended for new code.

Alternatives

Copy constructors, static factory methods, records with “withers”.

Record-style copy

```
record Point(int x, int y) {
    Point withX(int nx) { return new Point(nx, y); }
}
```

Adapter

Convert one interface into another interface expected by the client.

Object adapter

```
interface PaymentGateway {
    PaymentResult pay(Money amount);
}

// Existing library: incompatible API
class LegacyBankClient {
    BankResponse charge(long cents) { ... }
}

class BankAdapter implements PaymentGateway {
    private final LegacyBankClient bank;
    public PaymentResult pay(Money amount) {
        BankResponse r = bank.charge(amount.cents());
        return PaymentResult.from(r);
    }
}
```

Use when

You cannot change the existing class or external library.

Examples

- Legacy APIs
- Third-party SDKs
- Testing fakes
- Cloud provider abstractions

Agentic coding angle: adapters are useful boundaries where an AI agent can replace one provider without touching business logic.

Structural

Object Adapter

The adapter contains an instance of the adaptee and translates calls.

```
// Target expected by our application
interface ReportSource {
    List<Row> readRows(Path file);
}

// Existing class: incompatible API
class LegacyCsvReader {
    List<String[]> readFile(String filename) { ... }
}

// Object adapter: composition + delegation
class CsvObjectAdapter implements ReportSource {
    private final LegacyCsvReader reader;

    CsvObjectAdapter(LegacyCsvReader reader) {
        this.reader = reader;
    }

    public List<Row> readRows(Path file) {
        return reader.readFile(file.toString()).stream()
            .map(Row::fromArray)
            .toList();
    }
}
```

Structure

Client → Target interface →
Adapter → Adaptee instance

Strength

Very common in Java. It works with final classes, multiple adaptees, injected dependencies and tests.

Trade-off

One extra object and explicit forwarding methods.

Most Java adapters are object adapters because Java encourages composition and allows only single class inheritance.

Structural

Class Adapter

The adapter inherits from the adaptee and implements the target interface.

```
// Target expected by our application
interface ReportSource {
    List<Row> readRows(Path file);
}

// Existing implementation we want to reuse
class LegacyCsvReader {
    List<String[]> readFile(String filename) { ... }
}

// Class adapter: inheritance + interface implementation
class CsvClassAdapter extends LegacyCsvReader
    implements ReportSource {
    public List<Row> readRows(Path file) {
        return readFile(file.toString()).stream()
            .map(Row::fromArray)
            .toList();
    }
}
```

Structure

Client → Target interface → Adapter, which is also an Adaptee subclass

Limitation in Java

This works only if the target is an interface. Java cannot extend two concrete classes, so the classic multiple-inheritance class adapter is not directly possible.

When useful

Small wrappers around a non-final class when inheritance is acceptable and there is no need to swap the adaptee object at runtime.

Rule of thumb: prefer object adapter unless inheritance clearly simplifies the code.

Facade

Provide a simple interface to a complex subsystem.

Simplified API

```
class ReportFacade {  
    private final Parser parser;  
    private final Analyzer analyzer;  
    private final ExporterFactory exporters;  
  
    Path generateReport(Path csv, Format format) {  
        Dataset data = parser.parse(csv);  
        Analysis result = analyzer.analyze(data);  
        return exporters.create(format).export(result);  
    }  
}
```

Before

Client knows Parser, Analyzer, ExporterFactory, file paths, error handling.

After

Client calls one intention-revealing method.

Modern relevance

REST controllers, service layers, SDK clients, CLI commands, AI tool wrappers.

Decorator

Add responsibilities dynamically without subclass explosion.

Composable behavior

```
interface DataSource {
    byte[] read();
    void write(byte[] data);
}

class FileDataSource implements DataSource { ... }

class EncryptedDataSource implements DataSource {
    private final DataSource inner;
    public byte[] read() { return decrypt(inner.read()); }
    public void write(byte[] data) { inner.write(encrypt(data)); }
}

DataSource ds = new EncryptedDataSource(
    new CachedDataSource(
        new FileDataSource(path)));
```

Use when

Behavior can be layered: logging, caching, compression, encryption.

Java examples

InputStream wrappers
Servlet filters
Middleware chains

Risk

Too many wrappers can make debugging harder.

Composite

Treat individual objects and object groups uniformly.

Tree of tasks

```
interface Task {
    int estimateHours();
}

record CodingTask(String name, int hours) implements Task {
    public int estimateHours() { return hours; }
}

class Epic implements Task {
    private final List<Task> children = new ArrayList<>();
    public int estimateHours() {
        return children.stream()
            .mapToInt(Task::estimateHours)
            .sum();
    }
}
```

Use when

You model recursive structures: folders, menus, ASTs, UI components.

Modern Java angle

Records are good leaves; sealed interfaces can restrict valid component types.

Sealed composite

```
sealed interface Expr permits NumberExpr,
AddExpr {}
```

Composite example: files and directories

A directory contains both files and other directories. Clients treat all nodes uniformly.

```
sealed interface FsNode permits FileNode, DirectoryNode {
    String name();
    long size();
}

record FileNode(String name, long bytes) implements FsNode {
    public long size() { return bytes; }
}

final class DirectoryNode implements FsNode {
    private final String name;
    private final List<FsNode> children = new ArrayList<>();

    DirectoryNode(String name) { this.name = name; }
    public String name() { return name; }
    public void add(FsNode node) { children.add(node); }

    public long size() {
        return children.stream().mapToLong(FsNode::size).sum();
    }
}
```

Leaf

FileNode has no children.
Its size is directly known.

Composite

DirectoryNode stores a list
of FsNode, so a child can
be either a FileNode or
another DirectoryNode.

```
DirectoryNode src = new DirectoryNode("src");
src.add(new FileNode("App.java", 4_200));
src.add(new FileNode("Test.java", 2_900));

DirectoryNode project = new DirectoryNode("tap");
project.add(new FileNode("README.md", 1_200));
project.add(src);

System.out.println(project.size());
```

Proxy

Control access to another object.

Caching proxy

```
interface DocumentService {  
    Document load(String id);  
}  
  
class CachingDocumentService implements DocumentService {  
    private final DocumentService real;  
    private final Map<String, Document> cache = new  
        HashMap<>();  
  
    public Document load(String id) {  
        return cache.computeIfAbsent(id, real::load);  
    }  
}
```

Kinds

- Remote proxy
- Virtual/lazy proxy
- Security proxy
- Caching proxy
- Smart reference

Framework examples

- Spring AOP proxies
- ORM lazy loading
- RPC clients
- Mock objects

Risk

Proxy boundaries can hide latency, failures and transactions.

Observer

Define a one-to-many dependency so changes are automatically notified.

Simple observer/event bus

```
interface Listener<T> {  
    void onEvent(T event);  
}  
  
class EventBus<T> {  
    private final List<Listener<T>> listeners = new ArrayList<>();  
  
    void subscribe(Listener<T> l) { listeners.add(l); }  
  
    void publish(T event) {  
        for (Listener<T> l : listeners) {  
            l.onEvent(event);  
        }  
    }  
}
```

Classic use

Model notifies multiple views in MVC.

Modern use

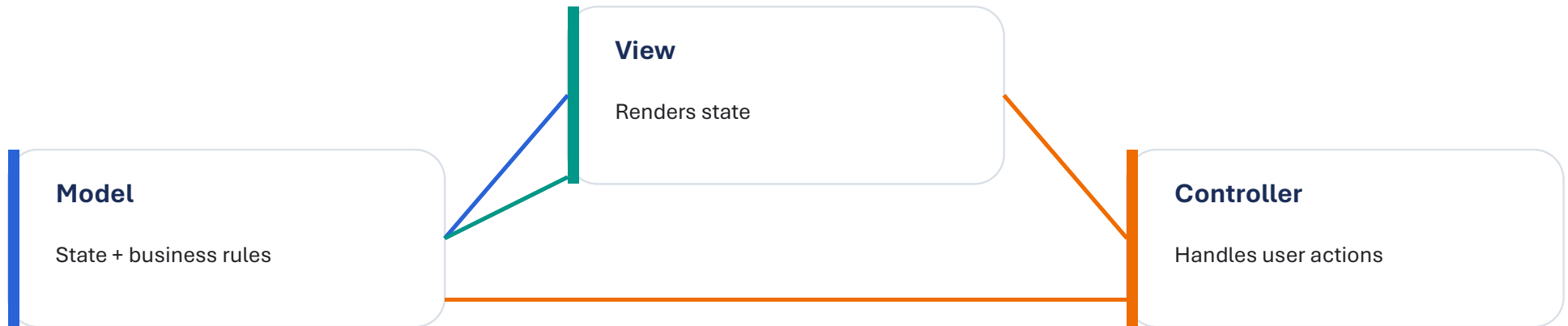
Events, streams, GUI listeners, reactive systems.

Risk

Implicit control flow: who reacts, when, and in what order?

MVC as a pattern composition

MVC combines separation of concerns with notification.



A model can have multiple views. When the model changes, views are notified and update themselves.

MVC + Observer in code

```
model.addListener(view::refresh); controller.onSave(() -> model.update(data));
```

Strategy

Encapsulate interchangeable algorithms behind the same interface.

Class-based strategy

```
interface PricingStrategy {  
    Money price(Order order);  
}  
  
class RegularPricing implements PricingStrategy { ... }  
class PremiumPricing implements PricingStrategy { ... }  
class BlackFridayPricing implements PricingStrategy { ... }  
  
class CheckoutService {  
    private final PricingStrategy pricing;  
    Money total(Order order) {  
        return pricing.price(order);  
    }  
}
```

Lambda-based strategy

```
PricingStrategy strategy = order ->  
    order.items().stream()  
        .map(Item::price)  
        .reduce(Money.ZERO, Money::add);
```

Use when

Many variants of an algorithm exist and the client should not know their details.

Command

Turn a request into an object.

Command object

```
interface Command {  
    void execute();  
    default void undo() { throw new  
        UnsupportedOperationException(); }  
}  
  
class CreateUserCommand implements Command {  
    private final UserService users;  
    private final UserData data;  
    public void execute() { users.create(data); }  
    public void undo() { users.delete(data.email()); }  
}
```

Use cases

Queues
Logs
Undo/redo
Schedulers
Transactions
Agent tool calls

Queueable requests

```
Queue<Command> queue = new ArrayDeque<>();  
queue.add(new CreateUserCommand(service,  
    data));  
queue.forEach(Command::execute);
```

Agentic programming: every tool invocation can be modeled as a command with inputs, outputs, validation and audit trail.

State

Change behavior when the internal state changes.

State objects

```
interface OrderState {  
    void pay(Order o);  
    void ship(Order o);  
    void cancel(Order o);  
}  
  
class DraftState implements OrderState {  
    public void pay(Order o) { o.setState(new PaidState()); }  
    public void ship(Order o) { throw new IllegalStateException(); }  
    public void cancel(Order o) { o.setState(new CancelledState()); }  
}
```

Use when

Large if/switch over state values controls behavior.

Benefit

Each state owns its valid transitions.

Alternative

Small state machines may be clearer with enum + switch.

Template Method

Define the skeleton of an algorithm and defer steps to subclasses.

Skeleton + hooks

```
abstract class DataImporter {  
    public final void importData(Path file) {  
        List<String> lines = read(file);  
        List<Record> records = parse(lines);  
        validate(records);  
        save(records);  
    }  
  
    protected abstract List<Record> parse(List<String> lines);  
    protected void validate(List<Record> records) { ... }  
}
```

Use when

The high-level algorithm is fixed but some steps vary.

Modern caution

Inheritance can be rigid. Strategy or composition may be simpler.

Composition alternative

```
parse = csvParser::parse;
```

Visitor

Add operations to an object structure without changing element classes.

Classic visitor + sealed model

```
sealed interface Expr permits Num, Add {  
    <R> R accept(ExprVisitor<R> v);  
}  
record Num(int value) implements Expr {  
    public <R> R accept(ExprVisitor<R> v) { return v.visit(this); }  
}  
record Add(Expr left, Expr right) implements Expr {  
    public <R> R accept(ExprVisitor<R> v) { return v.visit(this); }  
}  
interface ExprVisitor<R> {  
    R visit(Num n);  
    R visit(Add a);  
}
```

Use when

Stable data structure, many operations.

Examples

AST compilers
Interpreters
Static analyzers

Modern alternative

Pattern matching switch can replace simple visitors.

Iterator

Access elements sequentially without exposing representation.

Iterable<T> enables enhanced for

```
class Playlist implements Iterable<Song> {  
    private final List<Song> songs = new ArrayList<>();  
  
    @Override  
    public Iterator<Song> iterator() {  
        return songs.iterator();  
    }  
}  
  
for (Song s : playlist) {  
    player.play(s);  
}
```

Review connection

This reinforces Java review: Iterator<E>, Iterable<T>, enhanced for.

Modern extensions

Streams, Spliterator, reactive streams.

Functional iteration

```
playlist.stream().filter(Song::liked).to  
List();
```

Modern Java changes how we implement patterns

Some patterns become smaller with lambdas, records, sealed types and pattern matching.

Lambdas

Strategy, Command, Observer callbacks become functions.

Records

Small immutable products and DTOs become concise.

Sealed classes

Composite/Visitor hierarchies become closed and checkable.

Pattern matching

Some Visitor or State code becomes simpler.

Streams

Iterator/Strategy pipelines become declarative.

Annotations

Frameworks discover components and generate proxies.

Pattern compressed by the language

```
// Strategy without a class  
Comparator<User> byName = Comparator.comparing(User::name);
```

Dependency Injection: pattern behind many frameworks

DI is not one GoF pattern; it combines Factory, Strategy, Proxy and Inversion of Control.

Constructor injection

```
class ReportController {  
    private final ReportService service;  
  
    // dependency is provided from outside  
    ReportController(ReportService service) {  
        this.service = service;  
    }  
  
    ReportDto getReport(String id) {  
        return service.find(id);  
    }  
}
```

Annotated components

```
// Framework-style annotations  
@Service  
class CsvReportService implements  
    ReportService { ... }
```

Why it matters

- Testability
- Replaceable implementations
- Configuration outside business logic
- AI-safe extension points

Where patterns appear in real systems

Students should learn to recognize patterns in frameworks, not only toy examples.

MVC

Spring MVC, web frameworks, GUI frameworks

Observer

Event listeners, message buses, reactive streams

Proxy

Spring AOP, mocks, ORM lazy loading, remote clients

Factory

Dependency injection containers, ServiceLoader

Decorator

Streams, filters, middleware, interceptors

Command

Queues, jobs, undo/redo, CLI commands, agent tools

Design patterns in AI-assisted coding

Patterns give AI assistants stronger architectural constraints.

Weak prompt

“Add JSON export”
AI may add if-statements everywhere
Tests may be superficial
Architecture may degrade

Spec with pattern

“Add JsonExporter implementing Exporter”
“Register it in ExporterFactory”
“Do not modify PdfExporter”
“Add contract tests”

Agentic workflow

Plan
Modify code
Run tests
Review diff
Explain pattern impact

Architecture note for AI

Design decision: Use Strategy for Exporter implementations and Factory Method for selecting the exporter from CLI format.

Spec-driven assignment: apply one or more patterns

Students should justify design choices before coding.

Example proposal

```
# proposal.md

## Feature
Add HTML report export.

## Design decision
Use the existing Exporter Strategy interface.
Add HtmlExporter as a new implementation.
Use ExporterFactory to select it from --format html.

## Non-goals
Do not modify PdfExporter or JsonExporter.
```

Example tasks

```
# tasks.md

- [ ] Add HtmlExporter
- [ ] Register in ExporterFactory
- [ ] Add CLI test for --format html
- [ ] Add contract test for all exporters
- [ ] Update docs
- [ ] Explain pattern used
```

Anti-patterns: when patterns go wrong

Patterns should reduce complexity, not decorate it.

Pattern fever

Adding patterns where simple code is enough.

Singleton abuse

Global mutable state disguised as design.

Factory factory

Excessive indirection around simple constructors.

God object

One class coordinating everything.

Inheritance trap

Subclassing for reuse instead of composition.

AI architecture erosion

Small AI-generated changes bypass extension points.

A good design pattern makes the next change easier. If it only makes the current code more complicated, reconsider.

Which pattern should I consider?

Start from the design force, not from the catalog.

Object creation varies

[Factory Method / Abstract Factory / Builder](#)

Algorithm varies

[Strategy](#)

Many objects react to changes

[Observer / Publisher-Subscriber](#)

Request must be queued, logged or undone

[Command](#)

Need to wrap behavior dynamically

[Decorator / Proxy](#)

External API does not fit

[Adapter](#)

Complex subsystem needs simple entry point

[Facade](#)

Tree structures

[Composite](#)

Behavior depends on state

[State](#)

Summary

Design patterns are reusable design moves — not mandatory ceremony.

Remember

- Patterns capture design experience
- They communicate architecture
- They expose trade-offs
- They support reuse and testability

Apply carefully

- Start from a concrete problem
- Prefer simple code first
- Favor composition over inheritance
- Always discuss consequences

Modern angle

- Use Java language features
- Recognize framework patterns
- Guide AI with specs
- Protect design with tests

Final idea

A pattern is successful when the next change is local, explicit and testable.

References and source material

Use these sources for further study and examples.

Gamma, Helm, Johnson, Vlissides. Design Patterns: Elements of Reusable Object-Oriented Software.

Buschmann et al. Pattern-Oriented Software Architecture series.

Course repository: [pedrotgn/TAP](#) — use TAP examples as starting point for refactoring and exercises.

Modern Java documentation: records, sealed classes, pattern matching, streams, functional interfaces.